1 · DESIGN & NORMALIZATION

NF	Test (one line)	Smell	Fix
1NF	Every cell holds ONE value; no repeating groups; has a PK	`books='SQL, Java'` in one cell; columns `book1, book2`	One row per value, or move to a child table
2NF	1NF + no non-key column depends on only PART of a composite key	Key (order_id, book_id) but `title` depends on book_id alone	Move it to the table keyed by that part (`books`)
3NF	2NF + no non-key column depends on another NON-key column	`orders` stores customer_id AND customer email	Move to `customers`; keep only the FK

Memory: depends on the key (1NF), the whole key (2NF), and nothing but the key (3NF).

Relationships

Type	Example	How it is stored
1 : N	customers → orders	FK on the MANY side: `orders.customer_id`
M : N	students ↔ courses	Junction table `enrolments` with 2 FKs (= two 1:N)

```
-- junction table, composite PK
CREATE TABLE enrolments (
  student_id INT NOT NULL,
  course_id  INT NOT NULL,
  grade      VARCHAR(2),
  PRIMARY KEY (student_id, course_id),
  FOREIGN KEY (student_id)
    REFERENCES students(id),
  FOREIGN KEY (course_id)
    REFERENCES courses(id)
);
```

My labs: `id` PK + `UNIQUE KEY uq_student_course (student_id, course_id)` does the same job: the PAIR cannot repeat.

Keys

Key	Rule
PK	Unique + NOT NULL; one per table; indexed automatically
FK	Value must exist in the parent's PK (or be NULL if allowed); indexed automatically
UNIQUE	No duplicates; many per table; NULLs allowed (several)
Ref. integrity	No orphans. Child insert with no parent → Error 1452. Delete parent that has children → Error 1451 (RESTRICT) or children go too (CASCADE)

TRAPS · DESIGN

- FK lives on the MANY side, never on the "one" side.
- M:N cannot be stored with one FK: needs a junction table.
- Composite PK: the PAIR is unique; each column alone may repeat.
- Single-column PK + 1NF ⇒ already 2NF (2NF only bites with a composite key).
- UNIQUE ≠ PK: UNIQUE accepts NULL, PK never does.

2 · SQL QUERIES

```
SELECT major, COUNT(*) AS n    -- 5
FROM students                  -- 1
WHERE name LIKE 'S%'           -- 2 rows
GROUP BY major                 -- 3
HAVING COUNT(*) > 1            -- 4 groups
ORDER BY n DESC, major ASC     -- 6
LIMIT 3;                       -- 7
```

WHERE filters

`price BETWEEN 400 AND 700`	INCLUSIVE: ≥400 AND ≤700
`author IN ('Ben Forta','Martin Fowler')`	any of the list
`name LIKE 'P%'`	starts with P
`title LIKE '%SQL%'`	contains SQL
`name LIKE '_a%'`	2nd letter is a
`major IS NULL` / `IS NOT NULL`	never = `NULL`
`price > 500 AND stock <> 0`	AND before OR; use ( )

`%` = zero or more characters · `_` = exactly one · `DISTINCT` removes duplicate rows · ORDER BY default is ASC.

Aggregates and NULL

`COUNT(*)`	counts ROWS (NULLs included)
`COUNT(grade)`	counts non-NULL grades only
`COUNT(DISTINCT author)`	different non-NULL values
`SUM AVG MIN MAX`	skip NULL. AVG = SUM ÷ COUNT(col), not ÷ COUNT(*)
no rows at all	COUNT → 0; SUM/AVG/MIN/MAX → NULL

grades A, B, NULL → `COUNT(*)`=3, `COUNT(grade)`=2.

GROUP BY vs HAVING

	WHERE	HAVING
Filters	single ROWS	whole GROUPS
Runs	BEFORE grouping	AFTER grouping
Aggregates	**not allowed**	allowed

```
SELECT author, COUNT(*) AS num_books
FROM books
WHERE price > 300
GROUP BY author
HAVING COUNT(*) > 1;
```

Rule: every SELECT column is either in GROUP BY or inside an aggregate.

JOIN

	Rows returned
INNER (JOIN)	one per matching pair; unmatched rows vanish
LEFT	all INNER rows + each unmatched LEFT row once, right side = NULL

14 students, 14 enrolments, 6 students with none → INNER = **14** rows, LEFT = 14 + 6 = **20** rows. A left row with k matches appears k times.

```
SELECT c.name, b.title, o.quantity
FROM orders o
JOIN customers c ON c.id = o.customer_id
JOIN books b     ON b.id = o.book_id
WHERE c.name = 'Pim'
ORDER BY b.title;
```

```
-- count per student, zeros kept
SELECT s.id, s.name,
       COUNT(e.id) AS courses
FROM students s
LEFT JOIN enrolments e
       ON e.student_id = s.id
GROUP BY s.id, s.name;
```

Subqueries

```
-- IN: students who have an enrolment
SELECT name FROM students
WHERE id IN
  (SELECT student_id FROM enrolments);
-- EXISTS: same answer, row by row
SELECT name FROM students s
WHERE EXISTS (SELECT 1 FROM enrolments e
              WHERE e.student_id = s.id);
```

"Has none" (anti-join), 3 ways:

```
-- 1 NOT EXISTS (always safe)
SELECT name FROM students s
WHERE NOT EXISTS
  (SELECT 1 FROM enrolments e
   WHERE e.student_id = s.id);
-- 2 LEFT JOIN ... IS NULL
SELECT s.name FROM students s
LEFT JOIN enrolments e
       ON e.student_id = s.id
WHERE e.id IS NULL;
-- 3 NOT IN (only if no NULL in list)
SELECT name FROM students
WHERE id NOT IN
  (SELECT student_id FROM enrolments
   WHERE student_id IS NOT NULL);
```

TRAPS · QUERIES

- **LEFT JOIN + COUNT(*)** counts the NULL-filled row → 1 for "no match". `COUNT(e.id)` → 0.
- **NOT IN with a NULL** in the list returns NOTHING: `3 NOT IN (1, 2, NULL)` is UNKNOWN. Use NOT EXISTS.
- **WHERE cannot use aggregates** (`WHERE COUNT(*) > 1` ✗ → HAVING) **or SELECT aliases** (`WHERE n > 1` ✗).
- **BETWEEN is inclusive** at both ends.
- `= NULL` is never true → `IS NULL`.
- Condition on the RIGHT table in WHERE turns a LEFT JOIN into INNER; put it in ON.
- **UPDATE / DELETE without WHERE hits every row.**
- LIKE with no % or _ behaves like =.

3 · OBJECTS & CONSTRAINTS

```
CREATE TABLE books (
  id     INT AUTO_INCREMENT PRIMARY KEY,
  title  VARCHAR(200) NOT NULL,
  author VARCHAR(100) NOT NULL,
  price  DECIMAL(8,2) NOT NULL,
  stock  INT NOT NULL DEFAULT 0
);
CREATE TABLE customers (
  id    INT AUTO_INCREMENT PRIMARY KEY,
  name  VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL,
  UNIQUE KEY uq_customers_email (email)
);
CREATE TABLE orders (
  id INT AUTO_INCREMENT PRIMARY KEY,
  customer_id INT NOT NULL,
  book_id     INT NOT NULL,
  quantity    INT NOT NULL DEFAULT 1,
  FOREIGN KEY (customer_id)
    REFERENCES customers(id)
    ON DELETE CASCADE,
  CONSTRAINT fk_orders_book
    FOREIGN KEY (book_id)
    REFERENCES books(id)
    ON DELETE RESTRICT
);
```

`AUTO_INCREMENT`	Leave the column out of INSERT; MySQL gives the next number. Must be a key; one per table
`DEFAULT 0`	Used when the column is left out of INSERT
`NOT NULL`	Value required (Error 1048)
`UNIQUE`	Duplicate → Error 1062

ON DELETE (deleting a PARENT row)

RESTRICT (default)	Parent still has children → DELETE refused, Error 1451, nothing changes
CASCADE	Children are deleted automatically with the parent
SET NULL	Child FK becomes NULL (column must allow NULL)

No ON DELETE written (Lab 9, Lab 11) = RESTRICT. Lab 10 schemas use CASCADE.

ALTER TABLE

```
ALTER TABLE courses
  ADD COLUMN seats INT NOT NULL
  DEFAULT 30;
ALTER TABLE courses
  MODIFY COLUMN title VARCHAR(200)
  NOT NULL;
ALTER TABLE courses DROP COLUMN seats;
ALTER TABLE orders
  ADD CONSTRAINT fk_orders_book
  FOREIGN KEY (book_id)
  REFERENCES books(id);
ALTER TABLE customers
  ADD UNIQUE KEY
  uq_customers_email (email);
```

```
UPDATE books SET price = price * 1.10
WHERE id = 1;
DELETE FROM orders WHERE id = 7;
```

VIEW

```
CREATE VIEW order_details AS
SELECT o.id AS order_id, c.name,
       b.title, o.quantity,
       o.quantity * b.price AS total
FROM orders o
JOIN customers c ON c.id = o.customer_id
JOIN books b     ON b.id = o.book_id;

SELECT * FROM order_details
WHERE total > 1000;
DROP VIEW order_details;
```

- Stores the QUERY, not the data (a saved SELECT with a name).
- Always shows the CURRENT base tables: new order → appears in the view at once.
- Queried like a table. `CREATE OR REPLACE VIEW` changes it.

TRAPS · OBJECTS

- MODIFY replaces the WHOLE definition: restate NOT NULL / DEFAULT or they are lost.
- Create parent before child; drop child before parent.
- FK column type must match the parent PK (BIGINT ↔ BIGINT).
- Default ON DELETE is RESTRICT, not CASCADE.
- A view is not a snapshot or copy; it reruns its query each time.
- AUTO_INCREMENT numbers are not reused (gaps are normal).

4 · PROCEDURES · TRIGGERS · TRANSACTIONS

```
DELIMITER //
CREATE PROCEDURE place_order(
  IN p_customer INT, IN p_book INT,
  IN p_qty INT, OUT p_order_id INT)
BEGIN
  DECLARE v_stock INT;  -- DECLARE first
  SELECT stock INTO v_stock
    FROM books WHERE id = p_book;
  IF v_stock IS NULL THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'no such book';
  ELSEIF v_stock < p_qty THEN
    SET p_order_id = 0;  -- not enough
  ELSE
    UPDATE books
       SET stock = stock - p_qty
     WHERE id = p_book;
    INSERT INTO orders
      (customer_id, book_id, quantity)
    VALUES (p_customer, p_book, p_qty);
    SET p_order_id = LAST_INSERT_ID();
  END IF;
END //
DELIMITER ;
CALL place_order(1, 5, 2, @oid);
SELECT @oid;
```

`DELIMITER //`	So `;` can appear inside the body. `END //` ends it; then `DELIMITER ;`
`IN` / `OUT`	IN = value in (default). OUT = value back, caller passes `@var`
`DECLARE`	Local variable, TOP of BEGIN only; `SET v = ..` assigns
`SELECT..INTO`	Result → variable; ONE row only (no row → NULL)
`IF`	`IF..THEN` · `ELSEIF..THEN` · `ELSE` · `END IF;`
`LAST_INSERT_ID()`	AUTO_INCREMENT id of the last INSERT on this connection

Lab 11: `CALL count_students(1, @n); SELECT @n;` (@n = session variable)

Trigger

```
DELIMITER //
CREATE TRIGGER after_enrolment_insert
AFTER INSERT ON enrolments
FOR EACH ROW
BEGIN
  UPDATE courses SET seats = seats - 1
   WHERE id = NEW.course_id
     AND seats > 0;
  IF ROW_COUNT() = 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'course is full';
  END IF;
END //
DELIMITER ;
```

Event	OLD.col	NEW.col
INSERT	**no**	yes (new row)
UPDATE	yes (before)	yes (after)
DELETE	yes (deleted row)	**no**

- 6 kinds: BEFORE / AFTER × INSERT / UPDATE / DELETE.
- **BEFORE**: check or change the row (`IF NEW.quantity <= 0 THEN SIGNAL..`, `SET NEW.x = ..`). **AFTER**: update OTHER tables.
- `FOR EACH ROW`: body runs once per affected row.
- `SIGNAL SQLSTATE '45000'` = user error (Error 1644): cancels the statement that fired the trigger.
- Fires automatically, cannot be CALLed. `SHOW TRIGGERS;`

Transaction

```
START TRANSACTION;
UPDATE courses SET seats = seats - 1
 WHERE id = 1 AND seats > 0;
INSERT INTO enrolments
  (student_id, course_id) VALUES (3, 1);
COMMIT;       -- keep both
-- or ROLLBACK;  -- undo both
```

Atomicity	All statements happen, or none
Consistency	Valid state → valid state; constraints hold
Isolation	Others never see uncommitted changes
Durability	After COMMIT it survives a crash

TRAPS · PROGRAMMABILITY

- **A failed statement undoes only ITSELF.** Earlier statements stay until you ROLLBACK.
- No START TRANSACTION = autocommit: each statement is saved at once; ROLLBACK does nothing.
- INSERT trigger has no OLD; DELETE has no NEW.
- `ELSEIF` not `ELSE IF`; DECLARE must come first.

5 · NODE.JS (mysql2 + Express)

```
// db.js
const mysql = require('mysql2/promise');
const pool = mysql.createPool({
  host: 'localhost', user: 'root',
  password: '', database: 'lab11',
  connectionLimit: 10 });
module.exports = pool;
```

```
// SELECT → [rows, fields]
const [rows] = await pool.execute(
  'SELECT * FROM books WHERE author = ?',
  [req.query.author]);
res.json(rows);    // [] if none; rows[0]
// INSERT → [result]
const [result] = await pool.execute(
  'INSERT INTO books (title, author, '
  + 'price) VALUES (?, ?, ?)',
  [title, author, price]);
res.json({ id: result.insertId });
```

`pool.execute`	Prepared statement: SQL shape first, values sent separately. Use with user input
`pool.query`	Plain query; ? values escaped by the driver. Labs: SHOW, EXPLAIN
`?`	One per VALUE, array order. Not for table/column names
`const [rows]`	Result is `[rows, fields]`; take the first item
`result`	`.insertId` new AUTO_INCREMENT id · `.affectedRows` rows changed (0 = no match → 404)

Transaction pattern

```
const conn = await pool.getConnection();
try {
  await conn.beginTransaction();
  const [u] = await conn.execute(
    'UPDATE books SET stock = stock - ? '
    + 'WHERE id = ? AND stock >= ?',
    [quantity, book_id, quantity]);
  if (u.affectedRows === 0) {
    await conn.rollback();
    return res.status(409).json(
      { error: 'not enough stock' });
  }
  const [r] = await conn.execute(
    'INSERT INTO orders (customer_id, '
    + 'book_id, quantity) VALUES
(?,?,?)',
    [req.params.id, book_id, quantity]);
  await conn.commit();
  res.status(201).json(
    { order_id: r.insertId });
} catch (err) {
  await conn.rollback();
  res.status(500).json(
    { error: err.message });
} finally {
  conn.release();  // always runs
}
```

Express: where the data arrives

```
app.use(express.json()); // for req.body
POST /customers/3/orders?notify=1
body {"book_id": 5, "quantity": 2}
```

`req.params.id`	`'3'`	`:id` in the path
`req.query.notify`	`'1'`	after the ?
`req.body.book_id`	5	JSON body

fetch (front end)

```
const res = await fetch('/books');
const books = await res.json();
```

POST: `fetch(url, { method: 'POST', headers: {'Content-Type': 'application/json'}, body: JSON.stringify({ book_id: 5 }) })`

TRAPS · NODE

- **pool.execute is NOT inside the transaction** (the pool may use another connection). Every statement goes through `conn`.
- **Params arrive as strings**: `req.params.id`, `req.query.x` are `'3'` not 3 → `Number(..)`. Only a JSON body keeps numbers.
- No `conn.release()` → pool runs out, app hangs.
- Missing `await` → a Promise, not rows. `const rows =` without brackets → `[rows, fields]`.
- fetch needs TWO awaits; no throw on 404/500, check `res.ok`.

6 · SECURITY & PERFORMANCE

```
// UNSAFE: input glued into the SQL
const sql = "SELECT * FROM books "
  + "WHERE author = '" + author + "'";
// attack input:  ' OR '1'='1
// → WHERE author = '' OR '1'='1'
//   always true: returns EVERY row
// SAFE: ? placeholder, value separate
pool.execute('SELECT * FROM books '
  + 'WHERE author = ?', [author]);
```

Safe: the value is always DATA, never SQL.

Privileges (least privilege)

Data (CRUD)	SELECT, INSERT, UPDATE, DELETE → the APP user
Schema	CREATE, ALTER, DROP → admin only

```
CREATE USER 'university_app'@'localhost'
  IDENTIFIED BY 'app_password';
GRANT SELECT, INSERT, UPDATE, DELETE
  ON css326_lab10.*
  TO 'university_app'@'localhost';
REVOKE DELETE ON css326_lab10.*
  FROM 'university_app'@'localhost';
```

GRANT .. ON db.* **TO** user · REVOKE .. ON db.* **FROM** user · `SHOW GRANTS FOR user;` · app user running DROP → Error 1142.

Indexes

```
CREATE INDEX idx_books_author
  ON books (author);
SHOW INDEX FROM books;
```

- A SORTED copy of the column with pointers to rows: MySQL jumps to the match instead of reading every row.
- Automatic on PRIMARY KEY, UNIQUE, FOREIGN KEY.
- Cost: slower INSERT/UPDATE/DELETE, more disk.

Sargable = index usable (→ = rewrite as a range)

WHERE ..	Index?
`author = 'Ben Forta'` (bare column)	YES
`price > 500` · BETWEEN	YES
`title LIKE 'Course 9%'`	YES
`title LIKE '%SQL'`	**NO**
`LEFT(title, 8) = 'Course 9'` → `title LIKE 'Course 9%'`	**NO**
`price * 1.07 > 500` → `price > 500 / 1.07`	**NO**
`YEAR(order_date) = 2026` → `order_date >= '2026-01-01' AND order_date < '2027-01-01'`	**NO**

EXPLAIN SELECT ..

type	Meaning (best → worst)
`const`	PK / UNIQUE = value: 1 row
`ref`	Non-unique index, = match
`range`	Index range: > < BETWEEN, LIKE 'abc%'
`index`	Reads the WHOLE index (a full scan)
`ALL`	Full TABLE scan: every row (slow)
key	Index used; **NULL = none**
rows	ESTIMATED rows examined

TRAPS · SECURITY & PERFORMANCE

- **A function around an indexed column kills the index** (`LEFT(title,8)`, `YEAR(d)`) → type ALL.
- Leading `%` in LIKE cannot use the index.
- String concat is injectable with `query` AND `execute`; the `?` protects.
- type `index` is not "good"; more indexes = slower writes.

7 · EXAM SKILLS

```
FROM/JOIN → WHERE → GROUP BY → HAVING
  → SELECT → ORDER BY → LIMIT
```

Aliases are born in SELECT: usable in **ORDER BY only**, not WHERE / GROUP BY / HAVING.

Trace: 1 joined rows (count them) · 2 WHERE crosses out rows · 3 buckets · 4 aggregate (skip NULL) · 5 HAVING crosses out buckets · 6 SELECT · 7 sort.

Fill-in-the-blank keywords

`INSERT INTO .. VALUES`	`UPDATE .. SET .. WHERE`
`DELETE FROM .. WHERE`	`LEFT JOIN .. ON`
`GROUP BY .. HAVING`	`ORDER BY .. DESC`
`FOREIGN KEY .. REFERENCES`	`ON DELETE CASCADE`
`CREATE VIEW v AS SELECT`	`CREATE INDEX i ON t (c)`
`AFTER INSERT ON t`	`FOR EACH ROW`
`GRANT .. ON .. TO`	`REVOKE .. FROM`

Errors: 1062 duplicate · 1452 parent missing · 1451 parent has children · 1048 NULL · 1644 SIGNAL · 1142 denied · 1064 syntax.